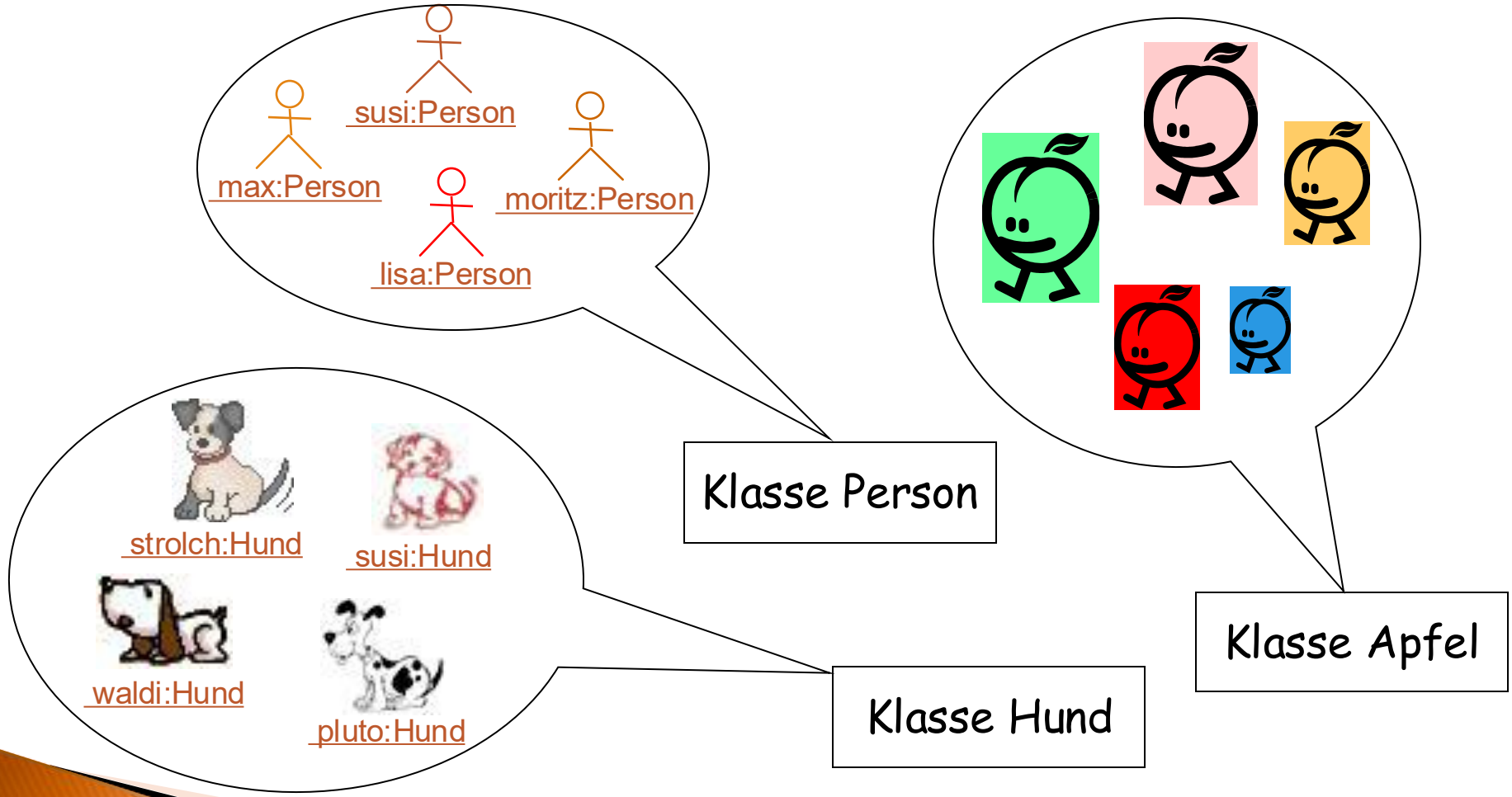
Recap

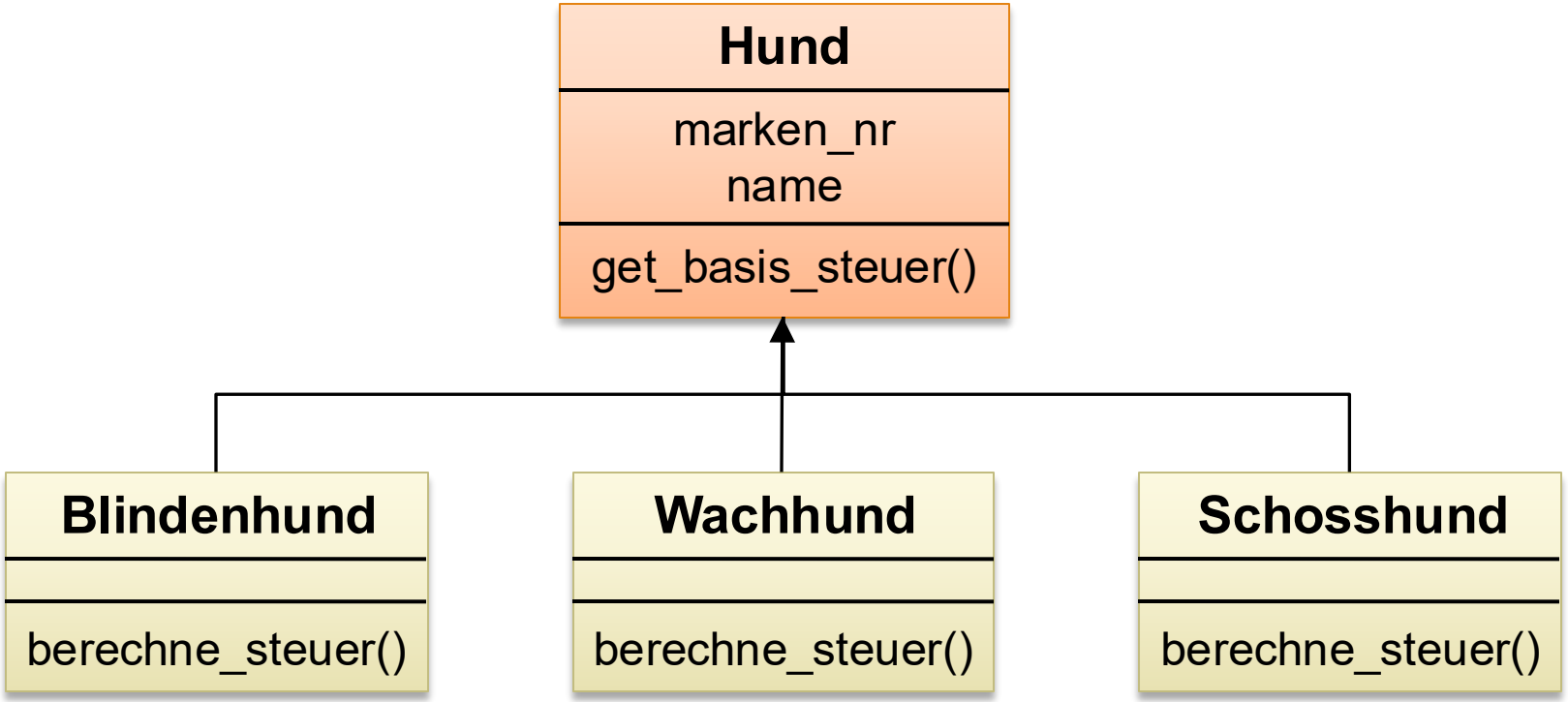
- Tag 1

Klassen und Objekte



Vererbung

- Beispiel "Hundesteuer berechnen"



Blindenhund, Wachhund und Schoßhund sind Spezialisierungen von Hund, mit jeweils eigener Berechnungsmethode

datetime

- Klasse für einen kompletten Zeitpunkt (Datum und Zeit)
  - Aktueller Wert: now() für date
  - Initialisierungen mit Parametern
  - Zugriff auf die einzelnen Komponenten: hour, minute, second bzw. day, month, year etc.

```
import datetime  # benötigtes Modul

jetzt = datetime.datetime.now()
print("Aktuell", jetzt)         # → "Aktuell 2026-10-20 11:48:19"
print("Stunden =", jetzt.hour) # → "Stunden = 11"
print("Jahr =", jetzt.year)    # → "Jahr = 2026"
```

Context Manager: with

```
# ohne Context Manager 'with'
file = open('file_path', 'w')
try:
  file.write('hello world')
finally:
  file.close()



# mit Context Manager 'with'
with open('file_path', 'w') as file:
  file.write('hello world !')
```

Exceptions

- `try`: Umschließt den "riskanten" Code, der eine Exception auslösen könnte.
- `except`: Behandelt spezifische Fehler, die im try-Block auftreten.
- `else`: Wird nur ausgeführt, wenn der try-Block keine Exception ausgelöst hat. Nützlich für "Erfolgs-Code", der nicht im try-Block stehen sollte.
- `finally`: Wird immer ausgeführt, egal ob eine Exception aufgetreten ist oder nicht. Ideal für Aufräumarbeiten (z.B. Schließen von Ressourcen).

Iterator

- In Python

Warum Iteratoren?

```
def get_all_logs():
  all_entries = db.query("SELECT * FROM logs")  # 1 Milliarde Zeilen
  return all_entries

# RAM-Problem: Versucht, 1 Milliarde Objekte *sofort* in den Speicher zu laden.
logs = get_all_logs()
for log in logs:
  # Verarbeitung beginnt erst, NACHDEM alles geladen ist
  process(log)
```

Das Iterator-Protokoll

- Python löst dies durch "Lazy Evaluation" (verzögerte Auswertung).
- Ein Iterator ist ein Objekt, das zwei Methoden implementiert:

1. **`__iter__()`**
   - Muss den Iterator selbst zurückgeben (`return self`).
   - Wird (implizit) von `for`-Schleifen aufgerufen.
2. **`__next__()`**
   - Gibt das **nächste** Element der Sequenz zurück.
   - Wenn keine Elemente mehr vorhanden sind, löst es die `StopIteration` Exception aus.

iter() & next()

- Python bietet zwei Built-in-Funktionen, um manuell mit dem Protokoll zu arbeiten:

```
my_list = [10, 20, 30]

# 1. iter() holt den Iterator von einem Iterable
my_iterator = iter(my_list)

print(type(my_list))     # <class 'list'>
print(type(my_iterator)) # <class 'list_iterator'>

# 2. next() ruft __next__ auf, um das nächste Element zu holen
print(next(my_iterator)) # Output: 10
print(next(my_iterator)) # Output: 20
print(next(my_iterator)) # Output: 30

# 3. Das Ende der Sequenz
print(next(my_iterator)) # Löst StopIteration aus
```

Iterable vs. Iterator

- **`Iterable`**
  - Ein Objekt, von dem man einen Iterator **bekommen** kann.
  - Implementiert nur `__iter__()`.
  - Beispiele: `list`, `tuple`, `dict`, `set`, `str`.
  - Man kann immer wieder `iter()` darauf aufrufen und erhält einen **neuen** Iterator (beginnt von vorn).
- **`Iterator`**
  - Das Objekt, das die Arbeit macht (das Zählen/Abrufen).
  - Implementiert `__iter__()` **und** `__next__()`.
  - Ist "Stateful" und erschöpft sich (Exhaustible).
  - Ein Iterator ist **auch** ein Iterable (da er `__iter__` hat), aber nicht umgekehrt!

Iterable vs. Iterator

```
# list ist ein Iterable
my_list = [1, 2]

# iterator_1 ist ein Iterator
iterator_1 = iter(my_list)
print(next(iterator_1)) # 1

# iterator_2 ist ein *neuer*, unabhängiger Iterator
iterator_2 = iter(my_list)
print(next(iterator_2)) # 1 (beginnt von vorn)

print(next(iterator_1)) # 2 (iterator_1 war bei Position 1)
```

Lazy Evaluation

- Iteratoren laden Daten „on demand“ – erst wenn `next()` aufgerufen wird.

```
# Datei-Handler in Python sind Iteratoren
with open("massive_log_file.log", "r") as f:
  # 'f' ist ein Iterator.
  # 'f.readline()' (oder __next__) wird bei jedem
  Schleifendurchlauf aufgerufen.

  for line in f:
    # Es ist immer nur EINE Zeile gleichzeitig im Speicher!
    process(line)

  # Ergebnis: Verarbeitung einer 100 GB Datei mit < 1 MB RAM.
```

Lab 5

Iteratoren

Higher-Order Functions

- lambda, map, filter, reduce

Wiederholung: Lambda

```
lambda arguments: expression
```

„Wegwerf-Funktionen“ für einfache, einmalige Operationen.

- **Anonym**: Sie haben keinen Namen (kein `def`).
- **Beschränkt**: Nur **eine** Anweisung, kein `return` (implizit).
- **Nützlich**: Perfekt als Argumente für Higher-Order Functions (wie `map`, `filter`, `sorted`).

Wiederholung: Lambda

```
# Traditionelle Funktion
def double(x):
    return x * 2

# Lambda-Äquivalent
double_lambda = lambda x: x * 2

# Nutzung
print(double_lambda(5)) # Output: 10
```

map(function, iterable)

- Wendet eine Funktion auf **jedes** Element eines Iterables an.
- 1:1 Transformation:
  `[x, y, z]` → `[f(x), f(y), f(z)]`

```
numbers = [1, 2, 3, 4]

# Nutzung mit lambda
doubled_iterator = map(lambda x: x * 2, numbers)

# WICHTIG: map() ist "lazy"
print(doubled_iterator)
# Output: <map object at 0x...>

# Um das Ergebnis auszugeben:
doubled_list = list(doubled_iterator)
print(doubled_list)
# Output: [2, 4, 6, 8]
```

filter(function, iterable)

- Filtert Elemente aus einem `Iterable`, basierend auf einer Bedingung.
- **N:M Selektion**: Die Funktion muss `True` oder `False` zurückgeben. Nur Elemente, für die sie `True` zurückgibt, bleiben erhalten.

```
numbers = [1, 2, 3, 4, 5, 6]

# Die lambda-Funktion ist das Prädikat (Bedingung)
even_iterator = filter(lambda x: x % 2 == 0, numbers)

# filter() ist ebenfalls "lazy"
print(even_iterator)
# Output: <filter object at 0x...>

even_list = list(even_iterator)
print(even_list)
# Output: [2, 4, 6]
```

reduce(function, iterable)

- Reduziert (aggregiert) ein `Iterable` auf einen einzigen Wert durch wiederholte Anwendung einer Funktion.

```
from functools import reduce

numbers = [1, 2, 3, 4]

# 1. (acc=1, el=2) -> 3
# 2. (acc=3, el=3) -> 6
# 3. (acc=6, el=4) -> 10
total = reduce(lambda acc, el: acc + el, numbers)

print(total)
# Output: 10
```

Lab 6

Map, Filter & Reduce

Generatoren

- mit Python

Generatoren: Funktionen

Funktion mit den folgenden Eigenschaften

- Statement `yield` statt `return`
- Automatische Bildung eines Iterator-Objektes, das den internen Zustand über mehrfache Aufrufe der Funktion hält
- Automatische Implementierung einer Methode `next()` und einer Exception `StopIteration`
- Vorteil: die Gesamtheit der Daten muß nicht im Speicher sein sondern nur das aktuelle Element: z,B. Lesen einer Datei

Generatoren: Funktionen

```
# Generator Funktion
def csv_reader(file_name):
  for row in open(file_name, 'r'):
    yield row.split(';')


# Verwendung der Funktion
for line in csv_reader('textfile.txt'):
  print(line)
```

Demo Generatoren

Lab 7

Generatoren

Decorator

- mit Python

Warum Decorators?

```
def process_data():
    start_time = time.perf_counter()  # --- SETUP ---

    # EIGENTLICHE LOGIK
    print("Starte Datenverarbeitung...")
    _ = [x ** 2 for x in range(10_000_000)]

    end_time = time.perf_counter()  # --- TEARDOWN ---
    print(f"process_data dauerte {end_time - start_time:.4f}s")


def fetch_web_data():
    start_time = time.perf_counter()  # --- SETUP (Identisch) ---

    # EIGENTLICHE LOGIK
    print("Starte Web-Download...")
    time.sleep(2)  # Simuliert Download

    end_time = time.perf_counter()  # --- TEARDOWN (Identisch) ---
    print(f"fetch_web_data dauerte {end_time - start_time:.4f}s")
```

Decorator

- Decorator erweitert die Funktionalität von bestehenden Funktionen, ohne diese Funktionen zu verändern
- Decorator Funktion ist ein Wrapper, der eine Vor- und Nachverarbeitung bei Funktionsaufrufen machen kann
- Decorator Klasse ist ein Wrapper für Funktionen, die einen internen Zustand als Klassenattribute verwalten kann (Zusätzlich zu der Vor- und Nachverarbeitung)
- Anwendungen:
  - Logging/Debugging
  - Zeitmessung
  - Webframework (Verarbeitung der Headers und Payloads)
  - Permissions abfragen
  - Caching

Decorator

```python
# 1. Der DECORATOR
def timer_decorator(func_to_wrap):
  # 2. Der WRAPPER
  def wrapper_function():
    start_time = time.perf_counter()

    func_to_wrap()  # 3. AUFRUF DER ORIGINALFUNKTION

    end_time = time.perf_counter()
    print(f"{func_to_wrap.__name__} dauerte {end_time - start_time:.4f}s")

  # 4. Der Decorator gibt den Wrapper zurück
  return wrapper_function


def process_data():
  """Eine Funktion, die dekoriert werden soll."""
  print("Starte Datenverarbeitung...")
  _ = [x ** 2 for x in range(10_000_000)]


# 5. Manuelle "Dekoration"
# Wir ersetzen die alte Funktion durch die neue Wrapper-Funktion
process_data = timer_decorator(process_data)

# 6. Aufruf des Wrappers
process_data()
```

Decorator @-Syntax

```
@timer_decorator
def process_data():
  """Eine Funktion, die dekoriert werden soll."""
  print("Starte Datenverarbeitung...")
  _ = [x ** 2 for x in range(10_000_000)]


# Equivalent
def process_data():
  """Eine Funktion, die dekoriert werden soll."""
  print("Starte Datenverarbeitung...")
  _ = [x ** 2 for x in range(10_000_000)]


process_data = timer_decorator(process_data)
```

Lab 8

Dekoratoren

Regular Expressions

- mit Python

Warum Regex?

- String-Methoden sind schnell, aber limitiert.

- Die Herausforderung
  - Wie findet man "Error 404" oder "Error 500"?
  - Wie extrahiert man *jede* E-Mail-Adresse aus einem Text?
  - Wie validiert man eine Telefonnummer, die (123) 456-7890 oder 123-456-7890 sein könnte?

- Methoden wie `find()`, `startswith()` oder `endswith()` versagen, sobald der zu findende Text einem **Muster** folgt, statt exakt gleich zu sein.

Das re-Modul

- Eine "Regular Expression" (Regex) ist eine Sequenz von Zeichen, die ein Suchmuster definiert. Python stellt dafür das re-Modul bereit.
- Verwenden Sie **immer** Raw Strings (`r"..."`) für Regex-Pattern.
- Regex verwendet Backslashes (\) für spezielle Sequenzen (z.B. **\d** für Ziffer).
- Python-Strings verwenden \ ebenfalls (z.B. **\n** für Zeilenumbruch).
  - Normaler String: `"\\d"`
  - Raw String: `r"\d"`

Das re-Modul

```python
import re

text = "Der Preis ist 100€."

# re.search() findet das ERSTE Vorkommen im gesamten String
# r"\d+" bedeutet "eine oder mehrere Ziffern (\d)"
match = re.search(r"\d+", text)

if match:
    print(f"Treffer gefunden: {match.group(0)}")
    # Output: Treffer gefunden: 100
```

Das re-Modul

- `re.search(pattern, string)`
  - Sucht nach dem **ersten** Vorkommen des Musters im String.
  - Gibt ein Match Object zurück (oder `None`)

- `re.match(pattern, string)`
  - Sucht nach dem Muster nur am Anfang des Strings.
  - *Seit Python 3.15 soft-deprecated zugunsten von `re.prefixmatch()`; in 3.12 unverändert.*

- `re.findall(pattern, string)`
  - Findet **alle** Vorkommen (die nicht überlappen).
  - Gibt eine **Liste von Strings** zurück.

- `re.sub(pattern, replacement, string)`
  - Sucht alle Vorkommen und **ersetzt** sie.

Regular Expressions

Zeichenklasse	Bedeutung
\d	Passt auf alle Ziffern des Dezimalsystems. Ist äquivalent zu [0-9].
\D	Passt auf alle Zeichen, die nicht Ziffern des Dezimalsystems sind. Ist äquivalent zu [^0-9].
\s	Passt auf alle Whitespace-Zeichen. Ist äquivalent zu [\t\n\r\f\v].
\S	Passt auf alle Zeichen, die kein Whitespace sind. Ist äquivalent zu [^ \t\n\r\f\v].
\w	Passt auf alle alphanumerischen Zeichen und den Unterstrich. Ist äquivalent zu [a-zA-Z0-9_].
\W	Passt auf alle Zeichen, die nicht alphanumerisch und kein Unterstrich sind. Ist äquivalent zu [^a-zA-Z0-9_].

Regular Expressions

Sonderzeichen	Bedeutung
\A	Passt nur am Anfang eines Strings.
\b	Passt nur am Anfang oder Ende eines Wortes. Ein Wort kann aus allen Zeichen der Klasse \w bestehen und wird durch ein Zeichen der Klasse \s begrenzt.
\B	Passt nur, wenn es sich nicht um den Anfang oder das Ende eines Wortes handelt.
\Z	Passt nur am Ende eines Strings.
^	Passt nur am Anfang eines Strings. Wenn das MULTILINE-Flag gesetzt wurde, passt ^ auch direkt nach jedem Newline-Zeichen innerhalb des Strings.*
$	Passt nur am Ende eines Strings. Wenn das MULTILINE-Flag gesetzt wurde, passt $ auch direkt vor jedem Newline-Zeichen innerhalb des Strings.*

Regular Expressions

Quantor	Bedeutung
?	Das vorangegangene Zeichen bzw. die vorangegangene Zeichenklasse darf entweder keinmal oder einmal vorkommen.
*	Das vorangegangene Zeichen bzw. die vorangegangene Zeichenklasse darf beliebig oft hintereinander vorkommen
+	Das vorangegangene Zeichen bzw. die vorangegangene Zeichenklasse darf beliebig oft hintereinander vorkommen, mindestens aber einmal.

Regular Expressions

Wiederholungen einer Zeichengruppe

Syntax	Bedeutung
{anz}	Das vorangegangene Zeichen bzw. die vorangegangene Zeichenklasse muss exakt anz-mal vorkommen.
{min,}	Das vorangegangene Zeichen bzw. die vorangegangene Zeichenklasse muss mindestens min-mal vorkommen.
{,max}	Das vorangegangene Zeichen bzw. die vorangegangene Zeichenklasse darf maximal max-mal vorkommen.
{min,max}	Das vorangegangene Zeichen bzw. die vorangegangene Zeichenklasse muss mindestens min-mal und darf maximal max-mal vorkommen.

Regular Expressions

Wiederholungen einer Zeichengruppe

Syntax	Bedeutung
()	Gruppierung (Zusammenfassung von Zeichen zu einer Einheit. z.B. (Python)+ => „Python“ kann ein oder mehrmals vorkommen
|	Alternativen z.B. P(ython|eter) => „Python“ oder „Peter“
(?i)	Extension (Flag): case insensitive
(?-i)	Extension (Flag): case sensitive

Regular Expressions

```
import re

print(re.findall(r'\bk[a-z]*', 'drei kleine katzen'))
# => Ausgabe ['kleine', 'katzen‘]

print(re.findall(r'\b(k[a-z]*n\b)', 'drei kleine katzen'))
# => Ausgabe ['katzen‘], Anfang und Ende wird gesucht

print(re.findall(r'\b(A\d{3}\w{1}\d{1})', 'A012B1 A01B1'))
# => Ausgabe ['A012B1‘], Anzahl wird überprüft

print(re.findall(r'\b(A\d{3}[BC]\d{1})', 'A012B1 A01B1 A456C2'))
# => Ausgabe ['A012B1', 'A456C2‘]

print(re.findall(r'\b(A\d{3}(DE|FG)\d{1})', 'A012DE1 A01B1 A456FG2'))
# => Ausgabe [('A012DE1', 'DE'), ('A456FG2', 'FG')]
```

Demo regex101.com

Performance 1

- `Findall` sucht *alles* und gibt eine **Liste von Strings** zurück
- `Finditer` gibt einen **Iterator** über **Match Objects** zurück

```
for match in re.finditer(r"ERROR: (.*)", logfile_content):
      # Es ist immer nur ein Match im Speicher
      print(f"Fehler gefunden: {match.group(1)}")
```

Performance 2

```
# Langsam:
for line in many_lines:
    # Regex wird 1000x neu kompiliert
    match = re.search(r"(\d{4}-\d{2}-\d{2})", line)


# Schnell:
date_pattern = re.compile(r"(\d{4}-\d{2}-\d{2})")

for line in many_lines:
    # 2. Das kompilierte Objekt wiederverwenden
    match = date_pattern.search(line)
```

Lab 9

Regular Expressions

Python-Entwicklung im Zeitalter von KI-Coding-Agenten

- Begriffe, Arbeitsablauf, Live-Demo

Warum dieses Kapitel?

- „Den Chat fragen“ und „einen Agenten arbeiten lassen“ unterscheiden sich in Vokabular und Arbeitsweise, nicht im Werkzeug.
- Typisches Scheitern: unscharfe Aufgabe hinein, selbstbewusster Unsinn heraus
- Ziel: ein Feature so spezifizieren, dass ein Agent es sauber baut und Tests es abnehmen

Begriff 1: Modell

- Die Parameter. Zustandslos: nimmt Token herein, sagt das nächste Token voraus
- Kann allein nichts: keine Datei lesen, keinen Befehl ausführen, kein Gedächtnis von gestern
- Stufen: `opus`, `sonnet`, `haiku`, `fable`
  schwer für Planung und Debugging, leicht für mechanische Änderungen
- Diagnose-Regel: „Das Modell ist schlecht“ stimmt selten. Erst Context und Harness prüfen.

Begriff 2: Harness

- Alles um das Modell, das daraus einen Agenten macht: Tools, System-Prompt, Context-Window-Verwaltung, Permissions, Hooks
- Agent-Loop: Modell schlägt vor, Harness führt aus, Ergebnis geht zurück, wiederholen
- Claude.ai und Claude Code laufen auf demselben Modell und verhalten sich anders: der Harness ist der Unterschied
- Ihre Konfiguration (`CLAUDE.md`, Permissions, Hooks) sind Anweisungen an den Harness

Begriff 3: Context

- Das, was der Agent gerade Relevantes weiß
  - nicht das Context Window (Token-Sequenz pro Anfrage), nicht die Session (gespeicherte Historie)
- Qualität statt Menge: volles Fenster mit altem Tool-Output = schlechter Context; eine einzelne Typdefinition = exzellenter Context
- Die meisten Alltagsfehler sind Context-Fehler: erfundene API, widersprochene Entscheidung, geratenes Schema. Erste Frage: Was war im Context?
- Context Engineering = kuratieren: `CLAUDE.md` für Dauerhaftes, Dateien gezielt laden, `/clear` zwischen Aufgaben

Begriff 4: Skill

- Eine lehrbare Fähigkeit als Paket: Ordner mit `SKILL.md` (Name, Beschreibung, Anleitung), optional Skripte und Vorlagen
- Nur Name und Beschreibung liegen im Context; der Rest wird geladen, wenn die Aufgabe passt (Progressive Disclosure)
- Offener Standard (agentskills.io), funktioniert über Harnesses hinweg
- Skill ≠ Tool: ein Tool wird aufgerufen, ein Skill wird gelesen.
  Skill ≠ `CLAUDE.md`: die wird immer geladen
- Aufruf in Claude Code: `/skill-name`

Zusammengefasst

- **Modell** = Gehirn: sagt Token voraus
- **Harness** = Hände und Regeln: führt aus, fragt nach, hält die Session
- **Context** = Schreibtisch: nur was gerade darauf liegt, zählt
- **Skill** = Handbuch im Regal: wird geholt, wenn die Aufgabe es braucht

Arbeitsablauf

- `/grill-me` → `/to-spec` | `/to-tickets` → `/implement` | `/implement-spec`
- Weiche: Passt die Arbeit in ein Context Window, reicht `/grill-me` → `/implement`. Sonst Spec und Tickets dazwischen.

Phase	Skill	Ergebnis	Wer entscheidet
Problem schärfen	`/grill-me`	Entscheidungen (keine Datei)	Mensch
Festhalten	`/to-spec` (ein Issue) oder `/to-tickets` (Tickets mit Abhängigkeiten)	Entscheidungsprotokoll	Agent schreibt, Mensch liest
Bauen	`/implement` (eine Session) oder `/implement-spec` (viele Tickets parallel)	Commit mit Tests	Agent

/grill-me: der größte Hebel

- Ein Interview in Runden: jede Frage mit Empfehlung, der Mensch entscheidet, Fakten holt der Agent selbst aus dem Repo
- Schreibt keine Dateien. Ergebnis ist eine scharfe Vorstellung im eigenen Kopf
- Warum der größte Hebel: fast jeder schlechte Agent-Output kommt aus einem unscharfen Problem-Space, nicht aus schlechtem Code
- Aktiv bleiben: zurückfragen, Scope beschneiden, „weiß ich nicht“ sagen. Passivität („agreed“ × 40) erzeugt selbstsicheren Unsinn
- Regeln: frische Session, kein Plan-Modus, aufhören, sobald Sie es präzise aufschreiben könnten

/to-spec und /to-tickets

- Spec = Entscheidungsprotokoll, weil Context Windows enden. Hält fest, was entschieden wurde, in der Sprache des Projekts
- Die Spec entscheidet nichts Neues. Steht etwas drin, das Sie nie gesagt haben, ist das ein Fehler der Spec
- Tickets = Tracer Bullets: jedes Ticket ein schmaler, vollständiger Pfad durch alle Schichten, allein vorführbar, mit Blocking-Kanten
- Faustregel: Tickets nur, wenn die Arbeit mehrere Sessions braucht

Bauen: /implement

- Baut nur Entschiedenes: kein Interview, keine Gegenvorschläge, keine neue Architektur
- Test-first an den Nahtstellen, Typecheck, Code-Review am Ende, Commit auf den aktuellen Branch
- `/implement-spec`: liest Tickets als Graph, arbeitet die Frontier parallel in Worktrees ab, ein Integrations-Branch, ein Pull Request
- Abnahme ist der Test, nicht „sieht gut aus“

Risiken und Permissions

- Erfundene Pakete: Paketnamen vor `uv add` auf pypi.org prüfen
- Geheimnisse im Context: `.env` per Deny-Regel vom Lesezugriff ausnehmen
- Prompt Injection über gelesene Dateien und Issues: Lesen ist nicht Befolgen
- Permissions: Startmodus ist `auto`. Zum Lernen bewusst in „Manual“ wechseln (Shift+Tab)
  - `--dangerously-skip-permissions` nur als Negativbeispiel

Demo: Überweisung mit KI-Agent

Lose Idee: „Kunden sollen Geld von einem Konto auf ein anderes überweisen können“

/grill-me → /to-spec → /implement auf lab_04/Lösung/account.py